Solution Design - Support Ticket Similarity Assistant

1. Problem

Solari receives about 150 support tickets a day. Agents need a few past tickets with reusable resolutions to reduce repeated investigation and inconsistent advice. The assistant retrieves evidence for an agent to review; it does not diagnose the new issue or apply a fix automatically.

2. Assumptions and rationale

Assumption	Rationale / consequence
Similar means a comparable symptom with a plausibly transferable fix, not merely the same category.	Related vocabulary alone cannot establish a shared root cause. Weak matches require investigation.
Search resolved tickets with nonblank resolution notes. Missing status means resolved, matching the import/API convention.	Unresolved tickets have no fix to reuse; a production importer must map helpdesk statuses explicitly.
The demo has 40 tickets. Assume up to one year of initial history: 150/day x 365, about 55k tickets. Threefold daily volume adds up to 165k next year, or about 220k including that backlog.	These are planning assumptions to confirm with Jamie, not measured customer data. Migrate before the prototype approaches 165k retained tickets.
Target p95 service latency below 300 ms, initially for one active request at a time.	Suggestions should appear when a ticket opens. Burst concurrency, HTTP overhead and production hardware still need validation.
English text, no attachments; one trusted local process.	Fits the supplied data and time box. Authentication, tenant isolation and PII controls precede deployment.
Precision matters more than recall; agents decide whether a past fix applies.	A misleading precedent can reinforce inconsistent advice. No-match is a valid outcome.

3. Implemented approach

```
CLI / HTTP API -> SimilarityService -> BM25F index [+ optional semantic index] -> verdict, matches, resolution notes
```

History is loaded from JSON at startup. Incremental resolved-ticket ingestion updates the in-memory indexes; it is not persisted. All relevant field types are checked across a batch before indexing, and indexed ticket fields are copied to prevent later caller mutations changing a displayed fix.

The default is dependency-free lexical retrieval: normalize phrases and common synonyms, remove boilerplate, stem lightly, and use BM25F with subject/tags/description/resolution weights of 3/2/1/0.4. Confidence is IDF-weighted query coverage, including unseen terms in the denominator; it is a heuristic, not a probability.

Optional hybrid mode embeds subject, description and tags with local MiniLM and combines lexical/dense top-50 rankings using RRF (k=60). Candidate confidence averages coverage and nonnegative cosine; a signal outside its top-50 pool contributes zero. This approximation needs evaluation on larger histories.

Matches retain retrieval order and must clear the weak threshold. The verdict describes the first displayed match, so a hidden candidate cannot upgrade it. `nearest` is that first match, or the top raw candidate when abstaining. Initial weak/strong thresholds are lexical 0.20/0.45 and hybrid 0.35/0.55. The sweep suggests weak thresholds only and never applies them automatically.

Architecture details: interactive diagram.

4. Evidence and limits

The 34-query development set contains 26 precedents and 8 issues without a reusable fix. Both modes rank the labelled precedent first for all 26 positives. After thresholding, lexical shows the correct first match for 25/26 and hybrid for 26/26; both abstain on 7/8 negatives. Both falsely accept NT-1 as a weak related match: an undelivered reset email is not solved by a password-case workaround. The supplied Snowflake example correctly abstains in both modes.

These are hand-written development labels, not held-out evidence. Hybrid recovers one lexical miss, but neither mode demonstrates reliable novel-issue detection. `eval --sweep` exposes the abstention/recall trade-off; agent-labelled holdout data is needed before changing policy thresholds.

The reproducible lexical benchmark and recorded environment are in the README. It measures single-request service calls on recombined seed text, with peak traced Python allocations during build. It does not establish a production worst case, concurrent capacity or hybrid scale performance. Treat 165k as a migration checkpoint, not a supported production capacity guarantee.

5. Trade-offs

- **Lexical vs embeddings vs LLM:** lexical is portable and explainable; hybrid helps vocabulary mismatch but adds model cost and still makes false matches. Ship the lexical prototype; trial hybrid after independent validation. An LLM is unnecessary for the required retrieval flow.

- **Exact vs approximate search:** exact cosine is simplest for this prototype. Benchmark realistic histories before choosing ANN; no million-vector latency guarantee is claimed.

- **Memory vs persistence:** rebuild-from-JSON keeps scope small, but loses incremental additions on restart. Before the planning envelope, move retrieval behind the service seam to persisted search, such as OpenSearch plus a vector store. Postgres FTS is another lexical option, not a drop-in BM25F equivalent.

- **Abstention vs always returning results:** filtering reduces misleading advice but hides useful weak precedents. Coverage thresholds and hand-maintained synonyms can overfit this tiny dataset. The starter overlap helper lacks weighting and abstention, so it was replaced.

6. More time, data or budget

First collect agent relevance feedback and a held-out set, including similar symptoms with different causes; measure handle time and answer consistency against a control queue. Next build the helpdesk sidebar/webhooks, persisted ingestion, authentication, PII controls and tenant isolation. Then add freshness/fix-version handling, duplicate grouping and burst-load tests. Consider reranking or a cited one-line LLM summary only after retrieval quality is established, with agent review retained.